

NS-3 NB-IoT Simulator User Guide

Introduction

This guide provides students with a fully prepared environment for running NS-3 NB-IoT simulations using Docker. The goal is to ensure a consistent, reproducible setup across all lab machines and student laptops.

The NS-3 simulator is packaged inside a Docker container, eliminating the need for manual installation of dependencies or compilation on your local system.

1. Overview	1
2. Installing Docker.....	2
Verify Installation	2
3. Project Structure	2
4. Building the NS-3 Simulator Container	2
5. Running the Container	3
6. Running NS-3 Simulations.....	3
7. Exiting and Re-entering the Container	4
8. Optional Cleanup	4
9. Final Example: Running nb-scenario1	4
Conclusion	5

1. Overview

The NS-3 NB-IoT simulator is provided as a Docker-based environment. Docker ensures that every student runs the same version of NS-3 with identical dependencies.

You will:

- Install Docker
- Build the NS-3 simulator container
- Run the container
- Execute NB-IoT simulations

2. Installing Docker

Docker allows applications to run inside isolated containers. Follow the official Docker installation guide:

<https://docs.docker.com/engine/install/>

If you prefer you can install its GUI (<https://docs.docker.com/desktop/>). In either case, choose your operating system (Windows, macOS, Linux) and follow the steps provided.

Verify Installation

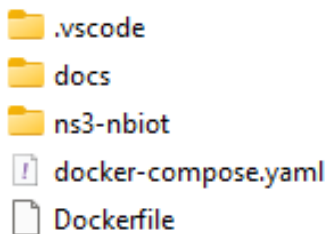
Open a terminal and run:

```
docker --version  
docker compose version
```

You should see version numbers printed.

3. Project Structure

Your instructor will provide a folder containing:



These files define the NS-3 simulator environment.

You do not need to modify them.

4. Building the NS-3 Simulator Container

Open a terminal and navigate to the project folder:

```
cd <project-folder>
```

Build the container:

```
docker compose build
```

This step:

- Downloads Ubuntu 22.04
- Installs NS-3 dependencies
- Clones the NB-IoT NS-3 repository
- Builds NS-3 using waf

The *build* may take several minutes and your computer needs to be connected to the Internet.

5. Running the Container

Start an interactive session inside the NS-3 environment:

```
docker compose run ns3
```

You will be placed inside the `workdir` defined in `docker-compose.yaml`.

By default, it is **`/opt/ns3-nbiot`**.

This directory contains the full NS-3 source tree.

There is another option to run the container and automatically remove it from the disk when you exit it. Use it with care, because the container is destroyed with no leftovers (you lose anything you left inside the container's disk), only the original image is intact.

```
docker compose run --rm ns3
```

6. Running NS-3 Simulations

Inside the container, simulations are executed using waf:

```
./waf --run <program-name>
```

For example, to run a scratch simulation:

```
./waf --run scratch/my-simulation
```

7. Exiting and Re-entering the Container

To exit the NS-3 environment:

```
exit
```

To re-enter later:

```
docker compose run ns3
```

8. Optional Cleanup

Remove the built image:

```
docker image rm ns3-simulator-ns3
```

Remove stopped containers.

```
docker container prune
```

If you run the container with the `--rm` option, every time you “exit” the container, it is automatically removed (and you cannot reenter it).

9. Final Example: Running nb-scenario1

Inside the container, run the NB-IoT scenario:

```
./waf --run scratch/nb-scenario1
```

The simulated scenario models a Narrowband IoT (NB-IoT) deployment using NS-3’s LTE/EPC framework, where a single base station (eNB) serves multiple IoT devices (UEs) distributed

randomly across a large cell. The devices communicate with a remote host through the EPC core network, generating sparse uplink traffic using UDP Echo applications to mimic typical IoT behavior. To create realistic network load, the simulation runs in three phases (Pre-Run, Run, and Post-Run) so that the main group of UEs operates under ongoing channel activity rather than an empty cell. A Winner+ propagation model¹ provides urban macro radio conditions, and each UE in the main phase is equipped with a Li-ion battery and a custom NB-IoT energy model to measure energy consumption during transmission. Overall, the scenario evaluates NB-IoT performance, radio behavior, and device energy usage under realistic cellular congestion.

If the simulation runs *successfully*Page, output will appear in the terminal.

```
root@fb6afb9332d5:/opt/ns3-nbiot# ./waf --run scratch/nb-scenario1
Waf: Entering directory `/opt/ns3-nbiot/build'
Waf: Leaving directory `/opt/ns3-nbiot/build'
Build commands will be stored in build/compile_commands.json
'build' finished successfully (4.520s)
Remote host address: 1.0.0.2
PGW address:
* Interface #0: 127.0.0.1
* Interface #1: 7.0.0.1
* Interface #2: 14.0.0.5
* Interface #3: 1.0.0.1
Node#0 Position:1526.82,80.4255,1.5
Node#0 Position:1394.71,573.577,1.5
Node#0 Position:468.124,696.703,1.5
Started computation at Sun Jul 19 15:24:22 2026
Number of UEs: 1 at each stage
Log dir: Finished computation at Sun Jul 19 15:25:13 2026
```

Conclusion

You now have a fully functional NS-3 NB-IoT simulation environment using Docker. This setup ensures consistency across all student machines and simplifies the process of running simulations during lab sessions.

¹ Bian et al. "A WINNER+ based 3-D non-stationary wideband MIMO channel model." IEEE Transactions on Wireless Communications 17, no. 3 (2017): 1755-1767.